

Cloud & AWS 소개

+

SageMaker를 활용한
분류 모델

Cloud Computing이란?

정의

인터넷을 통해 IT 리소스(컴퓨팅, 스토리지, DB 등)를 필요에 따라 이용하는 서비스

- 인터넷을 통해 IT 리소스를 필요 시점에 제공받는 방식
- 서버, 스토리지, 데이터베이스, 네트워크, AI 서비스까지 포함
 - 직접 구축보다 서비스 형태로 소비하는 모델
- 인프라 운영 부담을 줄이고 빠른 실험과 확장을 가능하게 함

클라우드의 핵심 특성

온디맨드

필요한 순간
바로 생성

탄력성

사용량에 따라
자동 확장/축소

종량제

사용한 만큼만
비용 지불

관리형

운영 부담을 줄이고
서비스에 집중

온프레미스와 클라우드 비교

온프레미스

도입 방식

장비 구매 후 설치

비용 구조

초기 투자 중심

확장 방식

증설까지 시간 소요

운영 부담

직접 유지보수

클라우드

즉시 프로비저닝

사용량 기반 과금

자동 또는 빠른 확장

관리형 서비스 활용 가능

클라우드가 필요한 이유?

초기 인프라 투자 비용 절감

서비스 출시 및 실험 속도 향상

글로벌 확장과 고가용성 구성 용이

분석, 보안, AI 기능을 빠르게 도입 가능

클라우드 서비스 모델

IaaS

인프라 자원 제공

EC2
EBS
VPC

PaaS

개발과 배포에
집중할 수 있는
플랫폼 제공

Lambda
SageMaker

SaaS

완성된
애플리케이션을
서비스로 사용

웹 메일
협업도구
CRM

AWS (Amazon Web Services)

200+

서비스

30+

리전

32%

시장점유율

AWS 서비스 분류

컴퓨팅

EC2, Lambda, ECS

스토리지

S3, EBS, EFS

데이터베이스

RDS, DynamoDB

AI/ML

SageMaker, Comprehend

컴퓨팅 서비스 이해

EC2

가상 서버를 직접
구성하고 운영

운영 통제력 높음

Lambda

서버 관리 없이
코드 실행

이벤트 기반 서버리스

ECS

컨테이너 기반 애플
리케이션 실행

마이크로서비스 운영에 적합

스토리지 서비스 이해

S3

객체 스토리지

데이터 레이크
백업
로그 저장

EBS

블록 스토리지

EC2 시스템 디스크
및
DB 저장

EFS

파일 스토리지

여러 인스턴스가
공유 접근 가능

데이터베이스 서비스 이해

RDS

관리형 관계형 DB

트랜잭션 중심 시스템

DynamoDB

완전관리형 NoSQL

고확장, 저지연 워크로드

Aurora

클라우드 최적화 RDB

고성능과고가용성

AWS 글로벌 인프라 개념

Region

Region

지리적으로 분리된
AWS 운영 영역

Availability Zone

각 Region 안의 독립적인
데이터센터 집합

Multi-AZ

장애 대응과
고가용성 확보를 위한
배치 전략

데이터/AI 관점의 AWS 계층 구조

AI Services

Rekognition, Textract, Comprehend, Lex

ML Platform

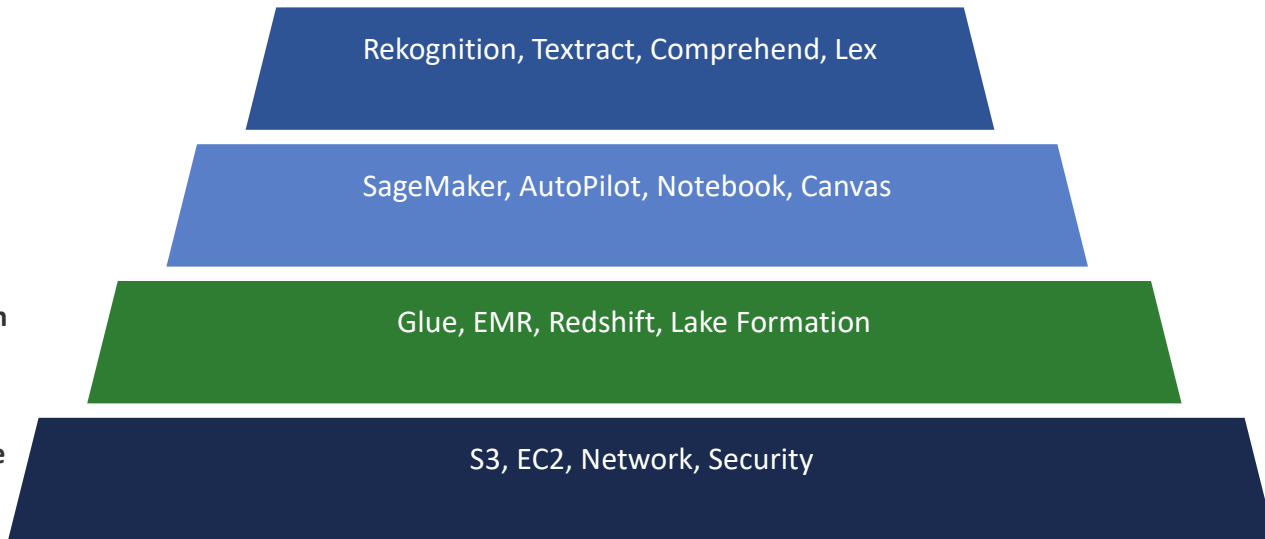
SageMaker, AutoPilot, Notebook, Canvas

Data Platform

Glue, EMR, Redshift, Lake Formation

Infrastructure

S3, EC2, Network, Security



AI Services와 ML Platform의 차이

AI Services

- 사전 학습된 기능 제공
- API 호출 중심 사용
- 빠른 도입 가능
- 운영 부담이 낮음

ML Platform

- 데이터 준비와 모델 학습 수행
- 튜닝과 배포 전략 직접 설계
- 높은 유연성 제공
- 맞춤화에 유리

보안과 비용을 함께 봐야 하는 이유

보안

IAM, 네트워크,
암호화, 로깅

비용

저장량, 호출 수,
전송량 기반 과금

운영

모니터링, 백업,
장애 대응 설계

거버넌스

표준화, 감사,
권한 통제

AWS AI/ML 서비스 스택

Application Layer

Forecast, Lookout, Personalize

AI Services

Rekognition, Textract, Comprehend, Lex

ML Platform

SageMaker (Canvas, AutoPilot, Notebook)

Infrastructure

S3, EC2 GPU, EMR, Glue

Machine Learning (기계 학습)

명시적 프로그래밍 없이 데이터로부터 패턴을 학습하고 예측하는 기술

ML 분류

지도 학습

회귀, 분류

비지도 학습

군집화, 차원축소

강화 학습

로봇, 게임 AI

분류 (Classification)

주어진 데이터를 정의된 클래스 중 하나로 예측하는 문제

분류 예시

- 이메일 분류: 스팸 / 정상
- 영화 흥행: 성공 / 실패 ← 오늘 실습!
- 신용 평가: 고위험 / 중위험 / 저위험

성능 평가 지표

정확도

전체 중 맞춘 비율

정밀도

양성 예측 중 실제 양성

재현율

실제 양성 중 맞춘 비율

Amazon SageMaker Canvas

No-Code ML 도구: 코딩 없이 드래그-드롭으로 모델 구축

주요 특징

 빠른 프로토타입 (2~3분)

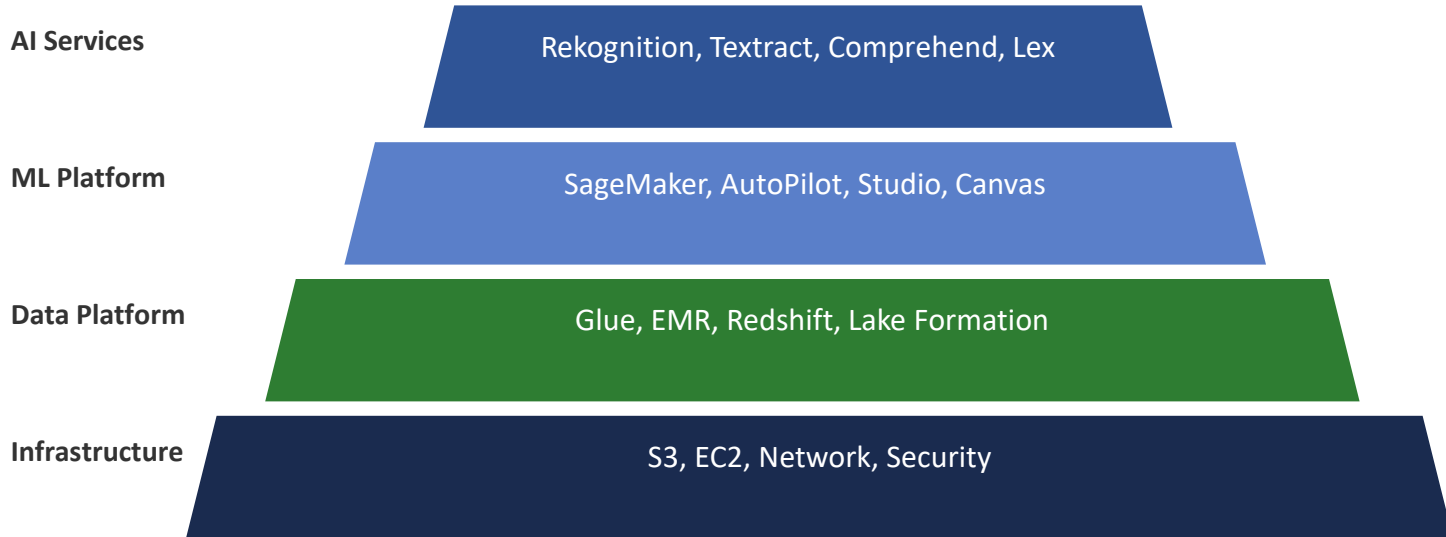
 자동 특성 엔지니어링

 모델 성능 자동 비교

 설명 가능성 (Feature Importance)

 예측 배치 처리 가능

AWS AI/ML 스택에서의 SageMaker 위치





개발 환경

JupyterLab
Studio 기반 작업

자동화

AutoPilot
파이프라인

현업 도구

Canvas
시각적 ML

운영

Endpoint
Monitoring

JupyterLab

- 코드 중심 개발 환경
- 데이터 과학자와 엔지니어에 적합
- 세밀한 제어 가능
- 재현성과 자동화에 강점

Canvas

- 시각적 No-Code/Low-Code 환경
- 현업 사용자와 분석가에 적합
- 빠른 프로토타이핑 가능
- 비즈니스 검증에 강점

정의

노트북, 코드, 터미널, 파일 브라우저를 통합한 인터랙티브 개발 환경

데이터 탐색부터 모델 실험과 배포 코드 작성까지 수행할 수 있는 SageMaker의 핵심 개발 워크스페이스

주요 특징

- Python 중심 실험 환경
- 노트북 기반 재현 가능성
- SDK/라이브러리 연동 용이
- 개발과 자동화의 연결점

탐색

데이터 로드와 EDA

Pandas, SQL,
시각화 라이브러리
활용

실험

특징 엔지니어링과 모델 학습

Scikit-learn,
XGBoost,
PyTorch 등 사용

운영 연계

배포 코드와 파이프라인 작성

SageMaker SDK,
Boto3,
CI/CD 연계

유연성

알고리즘,
라이브러리,
코드 구조를
자유롭게 선택

재현성

노트북과
스크립트로
실험 이력 관리 가능

확장성

대규모 데이터와
분산 학습으로
확장 가능

통합성

S3, SDK,
파이프라인,
배포와
자연스럽게 연결

학습 곡선

Python과
ML 개념
이해 필요

운영 책임

코드 품질과
환경 관리 필요

협업 규칙

노트북 표준화와
버전 관리 중요

비용 관리

인스턴스 사용 시간
관리 필요

정의

코딩 없이
데이터 불러오기,
문제 유형 선택,
모델 생성,
예측 결과 확인까지
시각적 인터페이스로 수행할 수 있는
SageMaker의
No-Code/Low-Code ML 환경

주요 특징

- 비개발자도 사용 가능
- 빠른 프로토타입
- 자동 특성 처리와 모델 비교
- 예측 결과를 시각적으로 확인



빠른 시작

**드래그-드롭과
마법사형 진행**

비개발자도 접근 가능

자동화

**자동 특성 처리와
모델 비교**

실험 속도 향상

설명 가능성

**중요 피처와
결과 해석 지원**

비즈니스
커뮤니케이션에 유리

속도

짧은 시간 안에
PoC 가능

접근성

현업 사용자도
스스로 실험 가능

협업

데이터팀과 현업팀의
공통 대화 기반 제공

가시성

성능과 결과를
직관적으로 확인

제어 한계

세밀한 알고리즘
제어는 제한적

복잡한 커스텀

특수한 전처리나
모델 구조에는 한계

운영 연결

고도화 시
개발팀 협업 필요

데이터 품질

입력 데이터
품질이 결과를
크게 좌우

JupyterLab 중심 접근

사용자

개발자, 데이터 과학자

작업 방식

코드 작성 중심

유연성

매우 높음

적합한 목적

정교한 실험, 자동화, 운영

Canvas 중심 접근

분석가, 현업 사용자

GUI 중심

제한적이지만 빠름

빠른 검증, 비즈니스 PoC

JupyterLab이 적합한 경우

- 커스텀 전처리 필요
- 배포 코드와 자동화가 중요
- 대규모 실험 관리 필요
- 개발팀 중심 프로젝트

Canvas가 적합한 경우

- 현업 아이디어를 빠르게 검증
- 코딩 역량이 제한적
- 시각적 결과 공유가 중요
- PoC 단계의 의사결정 지원



데이터

품질,
저장 위치,
접근 권한

사용자

누가 사용할 것인가

운영

배포,
모니터링,
재학습 전략

비용

인스턴스,
학습,
추론 비용 관리

1

SageMaker는 ML 생명주기 전체를 지원하는 AWS의 관리형 ML 플랫폼이다.

2

JupyterLab은 코드 중심의 유연한 개발 환경으로 정교한 실험과 운영 연계에 강하다.

3

Canvas는 No-Code/Low-Code 환경으로 빠른 PoC와 현업 중심 실험에 강하다.

4

실무에서는 두 도구를 역할에 맞게 조합하는 전략이 가장 효과적이다.

실습: TMDB 영화 흥행 예측

문제 정의

TMDB 영화 데이터를 학습하여 새로운 영화의 흥행 여부를 예측하는 분류 모델
랜덤 데이터와 TMDB 영화 데이터 2개를 사용해 각각 흥행 여부를 예측하는 분류 모델 생성

레이블 정의

성공 (1)

Accuracy(분류 정확도) 80% 이상

실패 (0)

위 조건 미충족

주요 활용 피쳐

budget (제작 예산) runtime (길이) imdb_score (평점)

release_month (개봉월) genre_count (장르) popularity (인기도)

실습 진행 단계

1 데이터셋 파일 생성

랜덤데이터 및 TMDB 데이터를 사용한 데이터셋
파일 생성

2 데이터셋 등록

생성한 데이터셋 등록

3 모델 생성 및 타겟 선택

모델 생성 및 success 컬럼 설정

4 피처 선택

입력 피처 검토 및 제외

5 모델 학습

Build 실행

6 성능 평가

정확도, 중요도 분석

Q&A

질문하고 배우세요!

이해가 잘 안 되는 부분이나 실습 중 막히는 부분이 있으면 언제든지 질문해주세요.